

Autograd Tensor Library & Inference Engine

A Beginner-Friendly Deep Dive into a From-Scratch C++20 Deep Learning Framework

Covers: Tensors · Computation Graphs · Backpropagation · OOP Design Patterns
Operations · Optimizers · nn Modules · Memory Management · Interview Q&A

Table of Contents

1	Introduction & Big Picture
2	What is a Tensor?
3	Object-Oriented Programming (OOP) Primer
4	Project Architecture & File Layout
5	The Tensor Class — Core Data Structure
6	The Operation Abstract Base Class
7	Differentiable Operations (Ops)
7.1	AddOp — Element-wise Addition with Broadcasting
7.2	MatMulOp — Matrix Multiplication
7.3	ReLUOp — Rectified Linear Unit
7.4	MSELossOp — Mean Squared Error Loss
8	The GraphEngine — Backpropagation
9	The MemoryPool — Memory Management (Singleton)
10	Optimizers
10.1	Base Class: Optimizer
10.2	SGD — Stochastic Gradient Descent
10.3	RandomizedCoordinateDescent
10.4	Adam — Adaptive Moment Estimation
11	Neural Network Module System (nn)
11.1	Module — Abstract Base
11.2	Linear Layer
11.3	ReLU Layer
11.4	Sequential Container
12	The flash:: API & Factory Functions
13	The Training Loop — Putting It All Together
14	Design Patterns Summary
15	Build System (Makefile)
16	How to Extend the Engine
17	Interview Q&A;

1. Introduction & Big Picture

This document walks through the **Autograd Tensor Library & Inference Engine**, a mini deep-learning framework written in modern C++20 — completely from scratch, without any external libraries like PyTorch, TensorFlow, or Eigen.

Think of it as a toy version of PyTorch. It can:

- Store numbers in multi-dimensional arrays called **Tensors**.
- Record every mathematical operation performed on Tensors in a **Computation Graph**.
- Automatically compute how the final answer (the loss) changes with respect to every parameter — this is called **Automatic Differentiation (Autograd)**.
- Use those changes (called **gradients**) to adjust parameters and improve the model's predictions — this is called **Optimization**.

■ Analogy: Imagine teaching a child to throw a ball at a target. After each throw, you measure how far off they were (loss), and give feedback about which direction and how much to adjust (gradient). The child gradually improves (optimization).

The entire framework is implemented using core C++ concepts: classes, inheritance, polymorphism, raw pointers, and the STL — making it a fantastic learning tool for both C++ OOP and machine learning internals.

2. What is a Tensor?

Before diving into code, you need to understand what a Tensor is — it's the fundamental building block of this entire library.

Scalars, Vectors, Matrices, and Tensors

Numbers come in different shapes:

Type	Dimensions	Example	Shape
Scalar	0-D	42.0	{}
Vector	1-D	[1, 2, 3]	{3}
Matrix	2-D	[[1,2],[3,4]]	{2,2}
Tensor	N-D	Any shape	{d0,d1,...}

In this project, a Tensor can be a 1-D vector (e.g., a bias vector of length 8) or a 2-D matrix (e.g., a weight matrix of shape {2, 8} meaning 2 rows and 8 columns).

Row-Major Memory Layout

Internally, ALL tensor data is stored in a single flat `std::vector<float>`. For a 2D matrix with *rows* rows and *cols* columns, the element at row *r*, column *c* is stored at index:

```
flat_index = r * cols + c
```

This is called **row-major** or C-style layout. For example, a 2×3 matrix:

```
[[1, 2, 3],      stored as: [1, 2, 3, 4, 5, 6]
 [4, 5, 6]]      element (1,2) = data[1*3 + 2] = data[5] = 6
```

■ Why flat storage? It's cache-friendly — accessing elements sequentially reads contiguous memory, which is much faster than jumping around.

3. Object-Oriented Programming (OOP) Primer

This project makes heavy use of OOP concepts. Here's a quick refresher:

Classes and Objects

A **class** is a blueprint. An **object** is an instance created from that blueprint.

```
class Dog {
public:
    string name;
    void bark() { cout << name << " says: Woof!\n"; }
};

Dog myDog; // object
myDog.name = "Rex";
myDog.bark(); // prints: Rex says: Woof!
```

Inheritance

One class can **inherit** (extend) another, gaining all its members and adding more.

```
class Animal {
public:
    virtual void speak() = 0; // pure virtual – must be overridden
};

class Dog : public Animal {
public:
    void speak() override { cout << "Woof!\n"; }
};

class Cat : public Animal {
public:
    void speak() override { cout << "Meow!\n"; }
};
```

Abstract Base Class (ABC) and Pure Virtual Functions

A class with at least one **pure virtual function** (written as = 0) is an **Abstract Base Class**. You cannot create an object of it directly — you must subclass it and implement all pure virtual functions.

Polymorphism

Polymorphism means "many forms". A base-class pointer can point to a derived-class object, and calling a virtual method will invoke the correct version automatically.

```
Animal* pet = new Dog(); // base pointer, derived object
pet->speak();             // prints: Woof! (dynamic dispatch)
```

Smart Pointers

`std::unique_ptr<T>` is an RAII wrapper: it owns a heap object and automatically deletes it when the pointer goes out of scope. This project uses `unique_ptr` in `Sequential` to own its layers.

4. Project Architecture & File Layout

Here is the complete file tree with a one-line description for each file:

```
Autograd-Tensor-Library-Inference-Engine/
■■■ include/                                (Header files – declarations)
■   ■■■ Tensor.h                          Core data structure
■   ■■■ Operation.h                      Abstract base for all ops
■   ■■■ GraphEngine.h                   Backpropagation driver
■   ■■■ MemoryPool.h                   Singleton memory manager
■   ■■■ Optimizer.h                     Optimizer ABC + SGD/Adam/RCD
■   ■■■ autograd.h                      Factory function API
■   ■■■ flash.h                         Master header (PyTorch-style API)
■   ■■■ ops/
■       ■ ■■■ Add.h                      Element-wise add op
■       ■ ■■■ MatMul.h                  Matrix multiply op
■       ■ ■■■ ReLU.h                    Activation op
■       ■ ■■■ MSELoss.h                 Loss function op
■   ■■■ nn/
■       ■ ■■■ Module.h                  Abstract layer base class
■       ■ ■■■ Linear.h                  Fully-connected layer
■       ■ ■■■ ReLULayer.h               ReLU activation layer
■       ■ ■■■ Sequential.h              Layer container
■■■ src/                                    (Source files – implementations)
■   ■■■ Tensor.cpp
■   ■■■ GraphEngine.cpp
■   ■■■ MemoryPool.cpp
■   ■■■ Optimizer.cpp
■   ■■■ ops/ (Add, MatMul, ReLU, MSELoss .cpp)
■   ■■■ nn/ (Linear, ReLULayer, Sequential .cpp)
■■■ main.cpp                             Demo training loop
■■■ Makefile                             Build script
```

The **include/** folder contains *declarations* (.h files) — telling the compiler what exists. The **src/** folder contains *implementations* (.cpp files) — the actual code. **main.cpp** ties everything together into a runnable demo.

Data Flow Diagram

Here is how data flows during a forward + backward pass:

```
User creates leaf Tensors (X, W, B)
|
v
factory functions (matmul, add, relu, mse_loss)
- allocate output Tensor on heap
- allocate Operation on heap
- call op->forward() [compute values]
- set output->creator = op [link into graph]
- register both with MemoryPool
|
v
GraphEngine::backward(loss)
- seed loss->grad = 1.0
- DFS post-order topological sort
- iterate reverse order, call op->backward()
|
```

```
      v
Gradients accumulated in W.grad, B.grad, etc.
      |
      v
optimizer.step() --> update W, B
      |
      v
MemoryPool::instance().cleanup() --> free intermediates
```

5. The Tensor Class — Core Data Structure

The **Tensor** class (in include/Tensor.h and src/Tensor.cpp) is the foundation of the entire engine. Every number — input data, weight, bias, intermediate activation, loss — is wrapped in a Tensor.

Member Variables

```
class Tensor {
public:
    std::vector<float> data;           // flat array of values
    std::vector<float> grad;          // gradient buffer (same length as data)
    std::vector<size_t> shape;        // e.g. {4, 2} means 4 rows, 2 cols
    Operation* creator;              // which Op produced this tensor? nullptr=leaf
    bool requires_grad;              // should we compute gradients for this?
    bool is_leaf;                    // was it created directly by the user?
};
```

The creator Pointer — Heart of the Graph

The most important field is creator. Every time a factory function (like matmul) creates a new Tensor from existing ones, it sets output->creator = op. This forms a chain of pointers linking the output back to its operation, and that operation back to its inputs, and so on — forming the Directed Acyclic Graph (DAG).

■ Leaf tensors (created directly, like your input data X or weights W) have creator == nullptr. This is how GraphEngine knows when to stop traversing.

Constructors

```
// 1. From existing data:
Tensor A({1.0f, 2.0f, 3.0f, 4.0f}, {2, 2}, /*requires_grad=*/true);
//   A.data  = [1, 2, 3, 4]
//   A.shape = {2, 2} => 2x2 matrix

// 2. Zero-filled:
Tensor Z({3, 4}, /*fill_value=*/0.0f, /*requires_grad=*/false);
//   Creates a 3x4 matrix of zeros
```

Element Access

```
Tensor A({1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f}, {2, 3});

float x = A.at(1, 2); // row=1, col=2 => index 1*3+2 = 5 => value: 6.0
A.at(0, 1) = 99.0f;   // modify element at row=0, col=1
```

Gradient Accumulation

Gradients are **added**, not overwritten, via accumulate_grad(). This is essential for graphs where a single tensor is used by multiple operations (a "diamond" shape). Each operation contributes its partial gradient, and the total is the sum of all contributions — as required by the chain rule.

```
void Tensor::accumulate_grad(const std::vector<float>& incoming_grad) {
    for (size_t i = 0; i < grad.size(); ++i)
        grad[i] += incoming_grad[i]; // ADD, not assign
}
```

Strides

For a tensor of shape $\{d_0, d_1, \dots, d_n\}$, the stride of dimension i is the product of all dimensions after it. For shape $\{2, 3\}$: $\text{strides} = \{3, 1\}$. This tells you: "to move one step along dimension 0 (rows), skip 3 elements".

6. The Operation Abstract Base Class

The **Operation** class (in `include/Operation.h`) is the abstract base class (ABC) for every differentiable mathematical operation in the engine.

```
class Operation {
public:
    std::vector<Tensor*> inputs;    // input tensors consumed by this op
    Tensor*                output;  // single output tensor produced

    virtual void forward() = 0;    // compute output from inputs
    virtual void backward() = 0;   // propagate gradients to inputs

    virtual ~Operation() = default;
    const std::string& name() const { return op_name; }

protected:
    std::string op_name = "Operation";
};
```

Key points:

- **forward()**: pure virtual — each subclass must implement how to compute the output.
- **backward()**: pure virtual — each subclass must implement how to compute and propagate gradients.
- **inputs / output**: raw pointers. The Operation does NOT own these Tensors — it just references them. Ownership is managed by MemoryPool.
- **Design Pattern: Strategy** — each Op encapsulates one math operation and its gradient. Adding a new op requires no changes to the engine core.

■ Because `forward()` and `backward()` are pure virtual, you can't create an 'Operation' object directly. You must create a subclass (like `AddOp`, `MatMulOp`). This enforces the contract: every op MUST provide both a forward and backward implementation.

7. Differentiable Operations (Ops)

Each concrete Operation subclass lives in `include/ops/` (header) and `src/ops/` (implementation). There are four: `AddOp`, `MatMulOp`, `ReLUOp`, and `MSELossOp`.

7.1 AddOp — Element-wise Addition with Broadcasting

File: `include/ops/Add.h` | `src/ops/Add.cpp`

Forward Pass

Two cases are supported:

- **Same shape** $[m,n] + [m,n]$: straightforward element-wise addition.
- **Broadcast** $[m,n] + [n]$: the shorter vector B is added to each row of A. This is used for adding a bias vector to every row of a batch.

```
// Same shape:
output[i] = A[i] + B[i]

// Broadcast ([m,n] + [n]):
output[r][c] = A[r][c] + B[c]    for all rows r
```

Backward Pass — Chain Rule

Let $dZ = \text{output} \rightarrow \text{grad}$ (the gradient of the loss with respect to the output Z).

```
// Gradient for A:
// Addition is linear => dZ/dA = 1 everywhere
dL/dA = dZ    (gradient passes straight through)

// Gradient for B (same-shape case):
dL/dB = dZ

// Gradient for B (broadcast case):
// B[c] was added to ALL rows of A, so its gradient is the SUM:
dL/dB[c] = sum over all rows r of dZ[r][c]
```

■ Why sum for broadcast? In the forward pass, $B[c]$ appeared in m different additions (one per row). By the chain rule, its gradient receives a contribution from each of those m outputs. Total = sum of all contributions.

7.2 MatMulOp — Matrix Multiplication

File: `include/ops/MatMul.h` | `src/ops/MatMul.cpp`

Three cases are handled at runtime based on the shapes of the inputs:

Case	Input Shapes	Output Shape	Use Case
A	$[m,n] \times [n,p]$	$[m,p]$	Batch $\times$ Weights
B	$[m,n] \times [n]$	$[m]$	Batch $\times$ Bias vec
C	$[n] \cdot [n]$	$[1]$	Dot product

Forward — Case A: Matrix $\times$ Matrix

```
// Z = A @ B where A is [m,n], B is [n,p], Z is [m,p]
Z[i,j] = sum over k of (A[i,k] * B[k,j])

// Code (k-loop in middle for cache reuse):
for (i): for (k): for (j):
    output[i*p+j] += A[i*n+k] * B[k*p+j];
```

Backward — Case A: Analytical Matrix Gradients

This is derived from calculus. Let L be the scalar loss.

$Z = A @ B$

$dL/dA = dL/dZ @ B^T$ (gradient w.r.t. A)
 $dL/dB = A^T @ dL/dZ$ (gradient w.r.t. B)

Derivation of dL/dA :

$Z[i,j] = \sum_k A[i,k] * B[k,j]$
 $dL/dA[i,k] = \sum_j (dL/dZ[i,j]) * B[k,j]$
 $= (dL/dZ @ B^T)[i,k]$ ✓

7.3 ReLUOp — Rectified Linear Unit

File: include/ops/ReLU.h | src/ops/ReLU.cpp

ReLU is the most popular activation function in deep learning. It zeroes out negative values and keeps positive values unchanged.

```
// Forward:
output[i] = max(0.0, input[i])

// Example:
input  = [-2.0, 0.5, -0.1, 3.0]
output = [ 0.0, 0.5,  0.0, 3.0]
```

The Mask Trick

During forward, a binary **mask** is saved:

```
mask[i] = 1.0f if input[i] > 0 // gate is open
mask[i] = 0.0f otherwise      // gate is closed
```

During backward, the gradient is simply multiplied by the mask:

```
dL/dinput[i] = dL/doutput[i] * mask[i]

// If the neuron was active, gradient flows through unchanged.
// If the neuron was dead ( $\leq 0$ ), gradient is blocked ( $= 0$ ).
```

■ Why save the mask? Because during backward we need to know which neurons were active during the FORWARD pass. We can't rerun forward during backward (could change state). Saving the mask is $O(N)$ memory but $O(1)$ extra time in backward.

7.4 MSELossOp — Mean Squared Error Loss

File: include/ops/MSELoss.h | src/ops/MSELoss.cpp

MSE measures how far the model's predictions are from the ground truth. The output is always a scalar (shape {1}).

```
// Forward:  
N    = number of elements in predictions  
loss = (1/N) * sum_i (pred[i] - target[i])^2
```

```
// Example (N=4):  
pred  = [1.0, 2.0, 3.0, 4.0]  
target = [1.0, 1.0, 1.0, 1.0]  
errors = [0.0, 1.0, 2.0, 3.0]  
loss   = (0 + 1 + 4 + 9) / 4 = 3.5
```

Backward

```
// Derivation:  
loss = (1/N) * sum_i e_i^2  where e_i = pred_i - target_i  
d(loss)/d(pred_i) = (1/N) * 2 * e_i = (2/N) * (pred_i - target_i)  
  
// With incoming gradient dLoss (= 1.0 at the root):  
grad_pred[i] = dLoss * (2/N) * (pred[i] - target[i])  
  
// Note: target has no gradient (it's a constant ground truth).
```

8. The GraphEngine — Backpropagation

File: include/GraphEngine.h | src/GraphEngine.cpp

The **GraphEngine** is the brain of the autograd system. It traverses the computation graph (the DAG built during forward) and propagates gradients backwards from the loss to all parameters.

The Three Steps of backward()

```
void GraphEngine::backward(Tensor* loss) {
    // Step 1: Seed — dL/dL = 1 by definition
    loss->grad.assign(1, 1.0f);

    // Step 2: Collect all operations in topological order
    std::vector<Operation*> order;
    std::unordered_set<Operation*> visited;
    if (loss->creator != nullptr)
        build_topo(loss->creator, order, visited);

    // Step 3: Execute backward in REVERSE topological order
    for (auto it = order.rbegin(); it != order.rend(); ++it)
        (*it)->backward();
}
```

Topological Sort — DFS Post-Order

A topological sort ensures that when we run backward() on an operation, all operations that depend on its output have ALREADY run their backward and contributed their gradient.

```
void GraphEngine::build_topo(Operation* op, ...) {
    if (visited.count(op)) return; // already seen (diamond graph)
    visited.insert(op);

    // First, visit all ancestor ops (go deeper into the graph)
    for (Tensor* inp : op->inputs)
        if (inp->creator != nullptr)
            build_topo(inp->creator, order, visited);

    // Then, append self (post-order means: children before parents)
    order.push_back(op);
}
```

Worked Example

Suppose we have: `loss = MSELoss(ReLU(Add(MatMul(X, W), B)), target)`

Forward graph (creation order):

```
MatMul(X, W) --> Z1
Add(Z1, B)   --> Z2
ReLU(Z2)     --> Z3
MSELoss(Z3)  --> loss
```

build_topo collects in POST-order:

```
[MatMulOp, AddOp, ReLUOp, MSELossOp]
```

Reversed (= execution order for backward):

```
[MSELossOp, ReLUOp, AddOp, MatMulOp]
```

Execution:

```
MSELossOp.backward() --> fills Z3.grad  
ReLUOp.backward()   --> fills Z2.grad (uses Z3.grad)  
AddOp.backward()     --> fills Z1.grad, B.grad (uses Z2.grad)  
MatMulOp.backward()  --> fills X.grad, W.grad (uses Z1.grad)
```

■ Why reverse? Each op's backward() reads output->grad (from downstream). That gradient must be fully computed before this op runs. Reversed topological order guarantees this because downstream ops always appear EARLIER in the reversed list.

9. The MemoryPool — Memory Management (Singleton)

File: include/MemoryPool.h | src/MemoryPool.cpp

Every training iteration creates dozens of temporary Tensor and Operation objects on the heap. Without cleanup, this is a memory leak. The MemoryPool solves this with a central registry.

The Singleton Pattern

A **Singleton** ensures only ONE instance of a class exists in the entire program. This is achieved using Meyers' Singleton (a static local variable):

```
class MemoryPool {
public:
    static MemoryPool& instance() {
        static MemoryPool pool; // created exactly once, thread-safe (C++11)
        return pool;
    }
    // Delete copy constructor and assignment to prevent copies:
    MemoryPool(const MemoryPool&) = delete;
    MemoryPool& operator=(const MemoryPool&) = delete;
private:
    MemoryPool() = default; // prevent external construction
    std::vector<Tensor*> tensors;
    std::vector<Operation*> ops;
};
```

Usage Pattern

```
// Inside a factory function (e.g., add()):
Tensor* out = new Tensor(...);
AddOp* op = new AddOp(a, b, out);
op->forward();
out->creator = op;
MemoryPool::instance().registerTensor(out); // track
MemoryPool::instance().registerOp(op);      // track
return out;

// At end of training iteration:
MemoryPool::instance().cleanup(); // free ALL tracked objects in one call
```

What Is Tracked vs What Is Not

- **Tracked (intermediate)**: outputs of factory functions (Z1, Z2, Z3, loss).
- **NOT tracked (leaf)**: user's tensors X, W, B, Y_target — they are stack-allocated or user-managed and must survive across iterations.

■ `cleanup()` must be called AFTER `optimizer.step()`, because `step()` reads gradients from intermediate tensors. Calling `cleanup()` before `step()` would free those tensors and cause undefined behavior (dangling pointer access).

10. Optimizers

File: include/Optimizer.h | src/Optimizer.cpp

After computing gradients, we need to update the parameters (W, B) to reduce the loss. This is the job of the optimizer. All optimizers share a common interface:

```
class Optimizer {
public:
    virtual void step()      = 0; // apply one update step
    virtual void zero_grad(); // set all gradients to 0
protected:
    std::vector<Tensor*> params; // the parameters to optimize
    float lr;                  // learning rate
};
```

10.1 SGD — Stochastic Gradient Descent

The simplest optimizer. The gradient tells us which direction INCREASES the loss, so we move in the OPPOSITE direction by a small step (learning rate η).

```
// Update rule:
param[i] = param[i] - lr * grad[i]

// Code:
void SGD::step() {
    for (Tensor* p : params)
        for (size_t i = 0; i < p->data.size(); ++i)
            p->data[i] -= lr * p->grad[i];
}
```

■ Analogy: Imagine standing on a hilly landscape (loss surface) blindfolded. You feel the slope under your feet (gradient) and take a step downhill (negative gradient). lr controls how big each step is — too large and you overshoot; too small and it takes forever.

10.2 RandomizedCoordinateDescent

Instead of updating ALL parameters at once, RCD randomly selects a subset (fraction of all coordinates) and updates only those.

```
// For each parameter tensor p:
k = max(1, fraction * p->data.size())
shuffle(all indices)
take first k indices
for each selected index i:
    p->data[i] -= lr * p->grad[i]
```

Useful when gradients are sparse or when the parameter space is very large.

10.3 Adam — Adaptive Moment Estimation

Adam is the most widely-used optimizer in deep learning. It maintains two per-parameter running averages ("moments") and adapts the learning rate individually for each parameter.

```
// At each step t, for each parameter:
m[i] = beta1 * m[i] + (1 - beta1) * grad[i] // 1st moment (mean)
```

```
v[i] = beta2 * v[i] + (1 - beta2) * grad[i]^2 // 2nd moment (variance)
```

```
// Bias correction (to avoid zero-bias at start):
```

```
m_hat = m[i] / (1 - beta1^t)
```

```
v_hat = v[i] / (1 - beta2^t)
```

```
// Parameter update:
```

```
param[i] -= lr * m_hat / (sqrt(v_hat) + eps)
```

```
// Defaults: beta1=0.9, beta2=0.999, eps=1e-8, lr=0.001
```

■ Why Adam is better than SGD: SGD uses the same learning rate for all parameters. Adam adapts per parameter — parameters with large gradients get smaller updates (v_hat is large, denominator is large), and sparse parameters get larger updates. This usually converges faster in practice.

11. Neural Network Module System (nn)

Files: include/nn/ and src/nn/

The nn module system mirrors PyTorch's torch.nn API. It provides a clean abstraction for building neural networks out of reusable layers.

11.1 Module — Abstract Base

```
class Module {
public:
    virtual Tensor* forward(Tensor* input) = 0;           // compute output
    virtual std::vector<Tensor*> parameters() = 0;        // return trainable params
    void zero_grad();                                     // zero all param grads
    virtual ~Module() = default;
};
```

Every layer MUST implement forward() and parameters(). The GraphEngine and optimizers interact with layers only through this interface (polymorphism).

11.2 Linear Layer

Implements: $Y = X \cdot W + B$

```
class Linear : public Module {
public:
    Linear(size_t in_features, size_t out_features, bool bias = true);
    Tensor* forward(Tensor* input) override;
    std::vector<Tensor*> parameters() override;

    Tensor W;    // weight matrix: shape {in_features, out_features}
    Tensor B;    // bias vector:   shape {out_features}
};

// Constructor initializes:
//   W: random uniform in [-0.1, 0.1], requires_grad=true
//   B: all zeros,                      requires_grad=bias

// forward() implementation:
Tensor* Linear::forward(Tensor* input) {
    Tensor* out = matmul(input, &W);    // X @ W
    if (use_bias) out = add(out, &B);    // + B (broadcast)
    return out;
}
```

■ Why random initialization? If all weights start at 0, every neuron computes the same value and gets the same gradient — the network can't learn different features. Small random values break this symmetry.

11.3 ReLULayer

A stateless layer that simply wraps the relu() factory function. It has no parameters (returns empty vector from parameters()).

```
Tensor* ReLULayer::forward(Tensor* input) {
    return relu(input); // just calls the factory function
}
```

```
}
```

11.4 Sequential Container

Holds an ordered list of layers and passes the output of each as input to the next. This is the **Composite** design pattern.

```
class Sequential : public Module {
    std::vector<std::unique_ptr<Module>> layers;
public:
    void add(std::unique_ptr<Module> layer); // append a layer

    Tensor* forward(Tensor* input) override {
        Tensor* out = input;
        for (auto& layer : layers)
            out = layer->forward(out); // chain layers
        return out;
    }

    std::vector<Tensor*> parameters() override {
        std::vector<Tensor*> all;
        for (auto& layer : layers) {
            auto p = layer->parameters();
            all.insert(all.end(), p.begin(), p.end()); // collect all
        }
        return all;
    }
};
```

12. The flash:: API & Factory Functions

File: include/flash.h (master header) and include/autograd.h (factory functions)

The **flash::** namespace is a thin convenience wrapper that lets users write PyTorch-style code in C++. It re-exports everything under a clean namespace:

```
namespace flash {
    using ::Tensor;
    using ::GraphEngine;
    using ::MemoryPool;
    using ::Adam;
    using ::SGD;
    using ::matmul;
    using ::add;
    using ::relu;
    using ::mse_loss;
}
```

Factory Functions

Factory functions in autograd.h encapsulate the boilerplate of allocating, computing, linking, and registering tensors and operations:

```
// Generic pattern for every factory function:
inline Tensor* add(Tensor* a, Tensor* b) {
    // 1. Determine output shape
    bool out_rg = a->requires_grad || b->requires_grad;
    // 2. Allocate output on heap
    Tensor* output = new Tensor(a->shape, 0.0f, out_rg);
    output->is_leaf = false;
    // 3. Allocate operation on heap
    AddOp* op = new AddOp(a, b, output);
    // 4. Run forward pass
    op->forward();
    // 5. Link into computation graph
    output->creator = op;
    // 6. Register both with MemoryPool
    MemoryPool::instance().registerTensor(output);
    MemoryPool::instance().registerOp(op);
    // 7. Return pointer to caller
    return output;
}
```

■ Design Pattern: Factory Function. The caller doesn't need to know about AddOp, MemoryPool registration, or graph linking. All that complexity is hidden behind the simple add(a, b) call.

13. The Training Loop — Putting It All Together

File: main.cpp

The demo trains a 2-layer MLP on a toy dataset (4 samples $\times$ 2 features) to predict 3 outputs. Let's walk through it step by step.

Step 1: Create Dataset

```
// Input: 4 samples, each with 2 features (shape: {4, 2})
flash::Tensor X({
    1.0f, 2.0f, // sample 0
    3.0f, 4.0f, // sample 1
    5.0f, 6.0f, // sample 2
    7.0f, 8.0f  // sample 3
}, {4, 2}, /*requires_grad=*/false);

// Target: 4 samples, each with 3 target values (shape: {4, 3})
flash::Tensor Y_target({
    1.0f, 0.0f, -1.0f, // target for sample 0
    2.0f, -1.0f, 0.0f, // target for sample 1
    -1.0f, 2.0f, 1.0f, // target for sample 2
    0.0f, 1.0f, -2.0f  // target for sample 3
}, {4, 3}, /*requires_grad=*/false);
```

Step 2: Build the Model

```
flash::Sequential model;
model.add(std::make_unique<flash::Linear>(2, 8)); // Linear(2 -> 8)
model.add(std::make_unique<flash::ReLULayer>()); // ReLU activation
model.add(std::make_unique<flash::Linear>(8, 3)); // Linear(8 -> 3)

// Network architecture:
// Input[4,2] -> Linear[2->8] -> [4,8] -> ReLU -> [4,8] -> Linear[8->3] -> [4,3]
```

Step 3: Create Optimizer

```
flash::Adam optimizer(model.parameters(), /*learning_rate=*/0.001f);
// model.parameters() returns: [W1, B1, W2, B2]
```

Step 4: Training Loop (100 steps)

```
for (int step = 0; step < 100; ++step) {
    // ① Zero gradients from previous iteration
    optimizer.zero_grad();

    // ② Forward pass: compute predictions
    flash::Tensor* pred = model.forward(&X); // shape: {4, 3}

    // ③ Compute loss (MSE)
    flash::Tensor* loss = flash::mse_loss(pred, &Y_target); // scalar

    // ④ Backward pass: compute all gradients
    flash::GraphEngine::backward(loss);
```

```
// ⑤ Update parameters using gradients
optimizer.step();

// ⑥ Free all intermediate tensors and ops
flash::MemoryPool::instance().cleanup();
}
```

■ The six steps above (zero, forward, loss, backward, step, cleanup) are the fundamental training loop in every deep learning framework. PyTorch, TensorFlow, and JAX all follow the same pattern — just with more features on top.

14. Design Patterns Summary

Pattern	Where Used	Why
Abstract Base Class / Polymorphism	Operation → AddOp, MatMulOp, etc.	GraphEngine can call backward() on any op without knowing its type.
Abstract Base Class / Polymorphism	Optimizer → SGD, RCD, Adam	Swap optimizers without changing the training loop.
Abstract Base Class / Polymorphism	Module → Linear, ReLULayer, Sequential	Uniform forward()/parameters() interface across all layers.
Composite	Sequential contains vector<	Nests arbitrary layers; forward() and parameters() recurse automatically.
Strategy	Each Op subclass owns forward math + gradient math	Adding a new op requires no changes to the engine core.
Singleton	MemoryPool	One global pool tracks and frees all intermediate allocations in one cleanup() call.
Factory Function	add(), matmul(), relu(), mse_loss() in autograd.h	Hides object wiring (alloc, forward, link, register) behind a clean API.

15. Build System (Makefile)

```
CXX      := g++
CXXFLAGS := -std=c++20 -Wall -Wextra -O2
TARGET   := autograd_engine

SRCS := src/Tensor.cpp src/GraphEngine.cpp src/MemoryPool.cpp \
        src/Optimizer.cpp src/nn/Linear.cpp src/nn/ReLULayer.cpp \
        src/nn/Sequential.cpp src/ops/Add.cpp src/ops/MatMul.cpp \
        src/ops/ReLU.cpp src/ops/MSELoss.cpp main.cpp

all: $(TARGET)

$(TARGET): $(SRCS)
    ■$(CXX) $(CXXFLAGS) -o $@ $(SRCS)

run:  $(TARGET)
    ■./$(TARGET)

clean:
    ■rm -f $(TARGET)
```

Commands:

- `make` — compile all `.cpp` files into the `autograd_engine` binary.
- `make run` — build and run the demo.
- `make clean` — delete the compiled binary.

16. How to Extend the Engine

Adding a New Operation (e.g., Sigmoid)

```
// Step 1: include/ops/Sigmoid.h
class SigmoidOp : public Operation {
public:
    SigmoidOp(Tensor* input, Tensor* output);
    void forward() override; // output[i] = 1 / (1 + exp(-input[i]))
    void backward() override; // dL/dinput[i] = dL/dout[i] * out[i]*(1-out[i])
private:
    std::vector<float> sigmoid_vals; // save for backward
};

// Step 2: src/ops/Sigmoid.cpp – implement forward and backward
// Step 3: add to autograd.h:
inline Tensor* sigmoid(Tensor* input) { /* same pattern as relu() */ }
// Step 4: add 'using ::sigmoid;' in flash.h
// Step 5: add src/ops/Sigmoid.cpp to SRCS in Makefile
```

Adding a New Layer (e.g., Dropout)

```
// Step 1: include/nn/Dropout.h
namespace flash {
class Dropout : public Module {
public:
    Dropout(float p = 0.5f); // p = drop probability
    Tensor* forward(Tensor* input) override;
    std::vector<Tensor*> parameters() override { return {}; } // no params
};
}

// Step 2: src/nn/Dropout.cpp
// Step 3: include in flash.h
```

17. Interview Questions & Answers

These questions are suitable for interviews covering C++, OOP, deep learning fundamentals, and system design, at a beginner-to-intermediate level.

C++ and OOP Questions

Q: What is an Abstract Base Class (ABC) in C++? How is it used in this project?

An Abstract Base Class is a class that has at least one pure virtual function (declared with '= 0'). You cannot instantiate it directly. In this project, `Operation` is an ABC with pure virtual `forward()` and `backward()`. Concrete subclasses like `AddOp` and `MatMulOp` must implement both. This enforces a contract: every operation **MUST** provide both a forward computation and gradient propagation. Similarly, `Module` is an ABC for neural network layers, and `Optimizer` is an ABC for parameter update algorithms.

Q: Explain the Singleton pattern. Why is `MemoryPool` a Singleton?

The Singleton pattern ensures only **ONE** instance of a class exists. In C++, Meyers' Singleton uses a static local variable inside a static member function. `MemoryPool` is a Singleton because we need exactly one central registry that tracks **ALL** heap-allocated intermediate tensors and operations across the entire program. If there were multiple pools, some objects would be missed during cleanup, causing memory leaks. Having one pool ensures `cleanup()` frees everything.

Q: What is polymorphism and how does `GraphEngine` use it?

Polymorphism means a base-class pointer can call overridden methods on derived objects. `GraphEngine` stores a list of `Operation*` (base pointers). When it calls `(*it)->backward()` in the loop, C++ automatically dispatches to the correct subclass — whether that's `AddOp::backward()`, `MatMulOp::backward()`, `ReLUOp::backward()`, etc. `GraphEngine` doesn't need to know which concrete type it has. This is the power of dynamic dispatch (virtual functions).

Q: What is `std::unique_ptr` and why does `Sequential` use it?

`std::unique_ptr` is a smart pointer that exclusively owns its heap object and automatically deletes it when the pointer goes out of scope (RAII). `Sequential` uses `vector<>` to own its layers. When a `Sequential` object is destroyed, all its layers are automatically deleted — no manual delete needed. Also, since `Module` is abstract (can't be copied), `unique_ptr` allows storing polymorphic objects safely.

Q: What does '`virtual ~Operation() = default`' mean and why is it important?

It declares a virtual destructor with the compiler-generated default behavior. A virtual destructor is **ESSENTIAL** when you have a base-class pointer to a derived object and you delete through that pointer. Without it, only the base destructor would be called, causing resource leaks in the derived class. Since `MemoryPool` stores `Operation*` pointers and calls 'delete op', the virtual destructor ensures the correct derived class destructor runs.

Q: What is a forward declaration and why is it used in `Tensor.h`?

A forward declaration tells the compiler that a class exists without giving its full definition. In Tensor.h, 'class Operation;' is a forward declaration. Tensor needs to store an Operation* pointer, but doesn't need the full definition just to hold a pointer. This breaks the circular dependency: if Tensor.h included Operation.h, and Operation.h included Tensor.h, the compiler would loop forever. Forward declarations solve this.

Data Structures & Memory

Q: What is row-major (C-style) memory layout? Why is it used here?

In row-major layout, a 2D matrix is stored row by row in a single flat array. Element (r, c) of an [m, n] matrix is at index $r * n + c$. It's used because: (1) std::vector is a single contiguous allocation — efficient cache usage; (2) sequential access patterns (like matrix multiplication inner loops) benefit from spatial locality; (3) it avoids pointer-chasing overhead compared to arrays of arrays (float**).

Q: Why does accumulate_grad() ADD to the gradient instead of overwriting it?

Because the same tensor can be used by multiple downstream operations (a 'diamond-shaped' graph). Each downstream operation computes a partial gradient contribution. By the chain rule, the total gradient is the SUM of all contributions. If we overwrote instead of accumulating, only the last contribution would survive, giving wrong gradients.

Q: What is the memory lifecycle of an intermediate Tensor in this engine?

1. A factory function (e.g., add()) allocates a Tensor with 'new' and registers it with MemoryPool. 2. The tensor is used in forward pass (values computed) and backward pass (gradients accumulated). 3. optimizer.step() reads gradients from leaf tensors (W, B). 4. MemoryPool::cleanup() calls 'delete' on all registered tensors. The tensor is gone. Leaf tensors (X, W, B) are user-managed and survive across iterations.

Machine Learning Concepts

Q: What is a computation graph (DAG)?

A Directed Acyclic Graph where each node is a Tensor and each edge is an Operation. In this project, the graph is built implicitly during the forward pass: each factory function sets output->creator = op, forming a chain of back-pointers. 'Directed' because gradients flow in one direction (output to inputs). 'Acyclic' because you can't have a tensor depend on its own output (no cycles).

Q: What is backpropagation and how does GraphEngine implement it?

Backpropagation is the algorithm for computing gradients of the loss with respect to all parameters using the chain rule. GraphEngine: (1) seeds loss->grad = 1 ($dL/dL=1$); (2) performs DFS post-order topological sort to collect all ops from loss back to inputs; (3) iterates in REVERSE topological order, calling each op's backward(). Reversed order ensures downstream gradients are ready before upstream ops run.

Q: What is the chain rule and how does MatMulOp use it?

The chain rule: if L depends on Z , and Z depends on A ($Z = A*B$), then $dL/dA = (dL/dZ) * (dZ/dA)$. For matrix multiplication $Z = A*B$: $dL/dA = dL/dZ @ B^T$ and $dL/dB = A^T @ dL/dZ$. These formulas propagate the gradient from the output Z back to both inputs A and B . `MatMulOp::backward()` implements exactly these formulas using nested loops.

Q: Why does ReLU save a mask during forward? What is a 'dead neuron'?

ReLU ($\max(0, x)$) is non-differentiable at $x=0$ but its subgradient is simple: 1 if $x>0$, 0 otherwise. The mask saves this binary gate so `backward()` can multiply the incoming gradient by it without re-running forward. A 'dead neuron' is one that always outputs 0 because its input is always negative — its gradient is always 0, so it never learns. This is a known issue with ReLU (sometimes mitigated with Leaky ReLU).

Q: What is MSE Loss and why is its output a scalar?

Mean Squared Error = $(1/N) * \sum((pred - target)^2)$. It measures the average squared difference between predictions and ground truth. It must be scalar because we need a single number to seed the backward pass with `grad=1`. 'Scalar' means shape {1}. During backward, the gradient of loss w.r.t. each prediction is $(2/N)*(pred-target)$, which is proportional to the prediction error.

Q: What is the difference between SGD and Adam optimizers?

SGD: `param -= lr * grad`. Uses the same fixed learning rate for all parameters. Simple but can be slow and sensitive to `lr` choice. Adam: maintains per-parameter running averages of gradients (1st moment = mean) and squared gradients (2nd moment = variance). Adapts the effective learning rate per parameter — large gradients get smaller updates, small gradients get larger updates. Bias correction prevents zero-bias early in training. Adam usually converges faster and is less sensitive to hyperparameter tuning.

System Design Questions

Q: Why are inputs stored as raw pointers in Operation instead of `std::shared_ptr`?

Performance: smart pointer overhead (reference counting, atomic operations) adds cost in the hot path. The engine uses an explicit ownership model: `MemoryPool` owns all intermediate Tensors and Ops. Raw pointers are valid for the lifetime of the computation graph (until `cleanup()`). Since there's a clear owner (`MemoryPool`) and a clear lifetime (one training iteration), raw pointers are safe and faster.

Q: Why must `MemoryPool::cleanup()` be called AFTER `optimizer.step()`?

The optimizer reads gradients from leaf tensors (W , B). Those gradients were accumulated by `backward()` calls on intermediate ops. If `cleanup()` were called before `step()`, the intermediate ops would be deleted. But more critically, if the leaf tensors happen to reference any intermediate results, those would be freed. The safe order is always: `zero_grad` -> `forward` -> `backward` -> `step` -> `cleanup`.

Q: How would you add a new differentiable operation to this engine? (e.g., Softmax)

1. Create `include/ops/Softmax.h` — declare class `SoftmaxOp` : public `Operation` with `forward()` and `backward()`.
2. Implement `src/ops/Softmax.cpp` — `forward` computes $\exp(x)/\sum(\exp(x))$, `backward` implements Jacobian of softmax times upstream gradient.
3. Add factory function in `autograd.h`

following the existing pattern (alloc, forward, link, register, return). 4. Add 'using ::softmax;' in flash.h. 5. Add src/ops/Softmax.cpp to SRCS in Makefile. No changes needed to GraphEngine, MemoryPool, or any existing op.

Q: What design pattern enables the GraphEngine to work with any Operation without knowing its type?

Polymorphism via virtual functions (also known as the Strategy pattern). GraphEngine holds a vector and calls (*it)->backward() in a loop. The virtual dispatch mechanism automatically calls the correct override — AddOp::backward(), MatMulOp::backward(), etc. This is Open/Closed Principle in action: the engine is closed for modification but open for extension (adding new ops).

Q: What is a topological sort and why is DFS post-order the right algorithm here?

A topological sort orders nodes in a DAG so that every node appears before the nodes it depends on. DFS post-order means: recurse into all children first, then add self. This naturally produces topological order: leaf ops (no dependencies) are added first, then ops that depend on them. When reversed for backward, we get the correct execution order where every downstream op runs before its upstream dependencies.

Q: How does the Composite pattern appear in the Sequential module?

Sequential is itself a Module (it implements forward() and parameters()). It contains a vector of Module objects (which can themselves be Sequential containers). This means you can nest models arbitrarily. forward() simply chains the layers, and parameters() recursively collects parameters from all sub-modules. The caller treats Sequential identically to any single layer — they don't need to know it's a container.

Closing Notes

This project is a miniature but complete deep learning framework built from first principles in C++20. Understanding it deeply will give you:

- A solid grasp of **automatic differentiation** — how PyTorch and TensorFlow compute gradients under the hood.
- Hands-on experience with core **OOP design patterns**: ABC, Polymorphism, Strategy, Singleton, Factory, Composite.
- Understanding of **memory management** in C++: heap allocation, raw pointers, RAI, and custom memory pools.
- Familiarity with **linear algebra** as it applies to neural networks: matrix multiplication, gradient derivation, chain rule.
- The ability to **extend the engine** with new operations, layers, and optimizers following established patterns.

Good luck in your journey into machine learning and C++ systems programming!